SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5117

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Assessment Tool Weightage: 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Debugging & Optimization Task

Code of Conduct:

I K.S.Dinesh(192324185) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Dinesh

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation
A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures.
B. Refactor the code to minimize redundant computations and memory usage.
C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation
A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation
A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic.
B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1) Performance Analysis of Block Cipher Modes of Operation

To debug and optimize a Python implementation of AES block cipher modes (ECB, CBC, CFB, and OFB), fix Padding and IV-handling errors.

A) Debug Padding and IV handling

* inCorrect Padding : AES requires data be processed in 16 bytes blocks in ECB and CBC modes.

* inCorrect IV handling : CBC, CFB and OFB require an initialization Vector (IV) of 16 bytes.

* inCorrect IV reuse : Generate a fresh, unpredictable IV for each message in CBC mode.

. Key matches : The encryption and decryption keys.

B) Optimized Python Program

```
pip install Pycryptodome

from Crypto.Cipher import AES

from Crypto.Random import get_random_bytes
```

```
from Crypto.Util.Padding import Pad, unpad
import tracemalloc
key = get_random_bytes(16)
Block_size = AES.block_size
```

How the Code is optimized:

1. A single AES key is reused for the experiment instead of generating a new key for every operation
2. The built-in Pad() and unpad()

c) Performance Comparison and optimization

Mode	Performance	Limitation
ECB	Blocks can be Processed	Reveals Patterns,
CBC	Encryption is sequential	Higher latency
CFB	Supports Streaming data	Sequential Processing
OFB	Generates a keystream	IV must never be reused.

The benchmark is a basic experiment, not a Complete Production security test.

Optimization and Security Analysis of Diffie - Hellman.

To implement the Diffie - Hellman Key exchange algorithm in Python, Correct shared - key generation errors, Optimize modular exponentiation.

A) Debugging shared key generation:

Alice's Public Key $A = g^a \bmod P$

Bob's Public key $B = g^b \bmod P$

Alice's shared key: $K_A = B^a \bmod P$

Bob's Shared Key: $K_B = A^b \bmod P$

Same key because g^{ab} mod $P = g^{ba} \bmod P$

Common errors and Solution:

1. incorrect modular arithmetic use Pow (base, exp)
2. incorrect exponent Values generate Private values

b) Optimized Python Program

```
bob_Private = secrets.randbelow(P-3)+2
start = time.Perf_Counter()
```

alice_Public = Pow(g, alice_Private, P)

bob_Public = Pow(g, bob_Private, P)

Optimization techniques:

use Pow(g, Private_key, P) rather than Calculating g**

Private_key %. P

Avoid Secrets instead of the ordinary random module for cryptographic private values.

c) Mitigating Man in the middle Attacks:

1. Digital signature: sign exchange Public key and verify them using trusted public keys.

2. Certificates: use Certificates to associate Public keys

3. key derivation: Pass the shared secret through a key derivation function such as HKDF before using it.

4. Ephemeral keys: Generate new private keys for each session to support forward secrecy.